

Lab 1: Basic Android Programming Report

學號：112511120 姓名：黃皓倫

1. 請說明 android 程式如何處理 widget 物件?

Widget 元件是 Android 畫面上的互動或顯示元件，例如 `TextView`，`Button`。Android 程式會先在 XML 版面配置檔中建立 Widget，並利用 `android:id` 給予每個元件唯一識別碼。接著在 `MainActivity` 的 `onCreate()` 中以 `setContentView()` 載入該版面，並透過 `findViewById(R.id.元件 id)` 取得 Widget 物件。取得物件後，程式即可設定元件的屬性或加入事件處理。例如實驗中，`demobutton` 被設定 `setOnClickListener()`；使用者按下按鈕時，程式便呼叫 `showtext.setText("Pass!")`，將 `TextView` 的文字更新為 `Pass!`。XML 負責定義介面與物件 ID，Java 程式則透過 ID 操作物件並回應使用者操作。

`onCreate()`：App 開啟、`MainActivity` 被建立時最先執行的重要方法。通常會在這裡載入畫面、找到元件，以及設定按鈕功能。

`setContentView()`：將 XML 設計好的畫面顯示在手機上。

2. 請說明 `R.id.button1` 是什麼?

`R.id.button1` 是 Android 自動產生的資源識別碼，用來代表 ID 為 `button1` 的 widget 元件。R 是 Android 根據專案內的資源檔自動建立的資源類別；`id` 表示元件 ID 的分類；`button1` 則是 XML 中透過 `android:id="@+id/button1"` 定義的按鈕名稱。程式可使用 `findViewById(R.id.button1)` 找到這個按鈕物件，並設定它被按下後要執行的功能。

3. 請舉例一個有結尾標籤的元件（可截圖）

```

<TextView
    android:id="@+id/showtext"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Android Lab1 Demo"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

```

結尾標籤表示這個 XML 元件到這裡結束，上方截圖的 `TextView` 裡面沒有再放其他元件，所以使用自我結尾的方式。

4. 請貼上自己的程式碼並附上註解

Demo1,2:

```

1. package com.example.lab1_1;
2.
3. import android.os.Bundle;
4. import android.util.TypedValue;
5. import android.view.View;
6. import android.widget.Button;
7. import android.widget.TextView;
8.
9. import androidx.appcompat.app.AppCompatActivity;
10.
11. public class MainActivity extends AppCompatActivity {
12.
13.     TextView showtext; // 宣告 TextView 物件變數 showtext
14.     Button demobutton; // 宣告 Button 物件變數 demobutton
15.     Button backbutton; // 宣告 Button 物件變數 backbutton
16.
17.     @Override
18.     protected void onCreate(Bundle savedInstanceState) {
19.         super.onCreate(savedInstanceState); // 呼叫父類別的 onCreate
           方法以執行預設初始化
20.         setContentView(R.layout.activity_main); // 設定 Activity 的
           畫面佈局內容為 activity_main.xml
21.
22.         showtext = (TextView) findViewById(R.id.showtext); // 透過
           ID 取得畫面中的 showtext TextView 物件
23.         demobutton = (Button) findViewById(R.id.demobutton); // 透
           過 ID 取得畫面中的 demobutton Button 物件
24.         backbutton = (Button) findViewById(R.id.backbutton); // 透
           過 ID 取得畫面中的 backbutton Button 物件
25.
26.         demobutton.setOnClickListener(new View.OnClickListener()
           { // 設定 demobutton 按鈕的點擊監聽器
27.             @Override

```

```

28.         public void onClick(View v) { // 當 demobutton 按鈕被點
           擊時執行的觸發事件
29.             showtext.setText("Pass!"); // 將 showtext 的顯示文字
           設定為 "Pass!"
30.             int size = (int) showtext.getTextSize(); // 取得 sho
           wtext 目前的文字大小
31.             showtext.setTextSize(TypedValue.COMPLEX_UNIT_PX, si
           ze + 10); // 將 showtext 的文字大小增加 10 像素
32.         }
33.     });
34.
35.     backbutton.setOnClickListener(new View.OnClickListener()
36.     { // 設定 backbutton 按鈕的點擊監聽器
37.         @Override
           public void onClick(View v) { // 當 backbutton 按鈕被點
           擊時執行的觸發事件
38.             showtext.setText("Android Lab1 Demo"); // 將 showte
           xt 的顯示文字恢復為 "Android Lab1 Demo"
39.             int size = (int) showtext.getTextSize(); // 取得 sho
           wtext 目前的文字大小
40.             if (size > 20) { // 判斷如果字型大小大於 20 像素
41.                 showtext.setTextSize(TypedValue.COMPLEX_UNIT_P
           X, size - 10); // 將 showtext 的文字大小減少 10 像素
42.             }
43.         }
44.     });
45. }
46. }

```

Demo3:

```

1. package com.example.lab1_2_112511120;
2.
3. import android.os.Bundle;
4. import android.view.View;
5. import android.widget.Button;
6. import android.widget.EditText;
7. import android.widget.TextView;
8.
9. import androidx.appcompat.app.AppCompatActivity;
10.
11. public class MainActivity extends AppCompatActivity {
12.
13.     TextView Text; // 宣告 TextView 物件變數 Text
14.     EditText editTextText; // 宣告 EditText 物件變數 editTextText
15.     Button buttonset; // 宣告 Button 物件變數 buttonset
16.     Button buttonreset; // 宣告 Button 物件變數 buttonreset
17.
18.     @Override
19.     protected void onCreate(Bundle savedInstanceState) {
20.         super.onCreate(savedInstanceState); // 呼叫父類別的 onCreate
           方法以執行預設初始化
21.         setContentView(R.layout.activity_main); // 設定 Activity 的
           畫面佈局內容為 activity_main.xml
22.

```

```

23.         Text = (TextView) findViewById(R.id.Text); // 透過 ID 取得畫面中的 Text TextView 物件
24.         editTextText = (EditText) findViewById(R.id.editTextText);
           // 透過 ID 取得畫面中的 editTextText EditText 物件
25.         buttonset = (Button) findViewById(R.id.buttonset); // 透過 ID 取得畫面中的 buttonset Button 物件
26.         buttonreset = (Button) findViewById(R.id.buttonreset); // 透過 ID 取得畫面中的 buttonreset Button 物件
27.
28.         buttonset.setOnClickListener(new View.OnClickListener() {
           // 設定 buttonset 按鈕的點擊監聽器
29.             @Override
30.             public void onClick(View v) { // 當 buttonset 被點擊時執行的事件
31.                 Text.setText(" Welcome to Android, " + editTextText.getText().toString()); // 將 Text 文字設定為歡迎詞加上輸入框的內容
32.             }
33.         });
34.
35.         buttonreset.setOnClickListener(new View.OnClickListener()
           { // 設定 buttonreset 按鈕的點擊監聽器
36.             @Override
37.             public void onClick(View v) { // 當 buttonreset 被點擊時執行的事件
38.                 Text.setText(" Hello World!"); // 將 Text 的顯示文字重設為 " Hello World!"
39.             }
40.         });
41.     }
42. }

```

5. 心得

實驗過程

首先是 demo1，把 activity_main.xml 照需求布置好，再照著投影片提示的 code 完成 MainActivity.java 就能過了。

MainActivity.java

```
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

public class MainActivity extends AppCompatActivity {
    TextView showtext;
    Button demobutton;    宣告變數型別、名稱

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        showtext = (TextView)findViewById(R.id.showtext);
        demobutton = (Button)findViewById(R.id.demobutton);
        demobutton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                showtext.setText("Pass!");
            }
        });
    }
}
```

Demo2 則是基於 demo1 的基礎修改。首先新增一顆按鈕之後，幫原本的 demobutton 在 setOnClickListener 裡面用 `int size = (int) showtext.getTextSize()` 取得當前 TextView 的大小，然後再用 `showtext.setTextSize(TypedValue.COMPLEX_UNIT_PX, size + 10)` 實現放大的功能；backbutton 用 `showtext.setText` 改變文字後，也使用 `showtext.setTextSize` 來縮小畫面上的文字。

Demo3 運用了新的元件 EditText，在 setOnClickListener 中用

```
public void onClick(View v) {
    Text.setText(" Welcome to Android, " +
        editTextText.getText().toString());
}
```

能夠讓 `EditText` 中由使用者輸入的文字被呼叫後顯示在 `Text` 中。

問題及解法

當一顆 `button` 被放到 `activity_main.xml` 上時，會立即報錯：

Missing Constraints in ConstraintLayout

This view is not constrained. It only has design-time positions, so it will jump to (0,0) at runtime unless you add the constraints

The layout editor allows you to place widgets anywhere on the canvas, and it records the current position with design-time attributes (such as `layout_editor_absoluteX`). These attributes are not applied at runtime, so if you push your layout on a device, the widgets may appear in a different location than shown in the editor. To fix this, make sure a widget has both horizontal and vertical constraints by dragging from the edge connections.

解決方法是對 `button` 右鍵>Constrain>然後把 `parent top`, `bottom`, `left` and `right` 都按過一遍。

會報錯的原因是 `tools:layout_editor_...` 屬性只會在 `Android Studio` 的預覽畫面中生效。在實體手機或模擬器執行時，因為沒有加上 `ConstraintLayout` 的約束屬性（如 `app:layout_...`），按鈕會跑掉並置頂在螢幕左上角 `(0, 0)`。